KeelNet: Controlled Abstention for Evidence-Grounded Question Answering

Naufal M. Kamaruddin Ideun Jeong Author 3

March 28, 2026

Abstract

KeelNet studies a narrower and more operational version of hallucination control: evidence-grounded question answering with an explicit answer-or-ABSTAIN decision. The current implementation now reports seven empirical checkpoints on SQuAD 2.0 with one question and one evidence passage as input. Stage 1 compares an answer-only baseline with an abstain-aware variant; the abstain-aware model reduces unsupported answers on the development split from 100.00% to 27.97% and improves decision-aware overall F_1 from 42.35 to 69.40, while answerable F_1 decreases from 84.82 to 66.76. Stage 2 adds a support verifier on top of the Stage 1 abstain-aware system. The verifier reaches 79.79 support F_1 on the development split at a selected support threshold of 0.25, but the resulting gated QA system changes end-to-end metrics only marginally: overall F_1 moves from 69.40 to 69.42, unsupported answer rate from 27.97% to 27.57%, and answerable F_1 from 66.76 to 66.41. Stage 2.5 then retrains the verifier with Stage 1 QA predictions as hard negatives. This raises verifier precision from 66.62% to 67.05% and lowers gated unsupported-answer rate from 27.57% to 26.95%, but overall F_1 slips from 69.42 to 69.34 and answerable F_1 from 66.41 to 65.62. Stage 3 then calibrates the original Stage 1 QA decision score and the original Stage 2 support score with scalar temperature scaling. On the development split, QA ECE falls from 0.2456 to 0.0753 and support ECE falls from 0.3326 to 0.1708; both fitted temperatures saturate at 5.0, which reveals strong overconfidence in the raw scores. Stage 4 then applies a fixed constrained controller over those calibrated QA and support signals. Its selected validation configuration satisfies the nominal 20% unsupported-answer budget with 14.30% unsupported-answer rate and 72.43 overall F_1, but on the development split it is effectively a null result: answerable F_1 remains 66.41, overall F_1 is 69.41, and unsupported-answer rate is 27.59%, which is almost identical to the original Stage 2 gate. Stage 5 then tests a first standalone support-constrained learner against these executed modular baselines. On the development split, this learner raises answerable F_1 to 71.40, but overall F_1 falls to 68.28 and unsupported-answer rate rises to 34.84%, missing its nominal 20% unsupported-answer budget. Stage 6 then tests a first adaptive revision of that learner. Relative to Stage 5, unsupported-answer rate falls only slightly from 34.84% to 34.45%, while answerable F_1 falls from 71.40 to 69.70 and overall F_1 falls from 68.28 to 67.62. Stage 7 then tests a first risk-budgeted action learner with explicit ABSTAIN as an action, separate utility and risk scores, and dual budget updates. At the selected risk threshold of 0.85, it satisfies both validation budgets with 19.22% unsupported-answer rate and 12.92% over-abstention rate, but the same operating point does not transfer cleanly to the development split: answerable F_1 reaches 69.95 and overall F_1 67.66, yet unsupported-answer rate remains 34.62%. The aggregated final comparison over Stage 1 and Stages 4–7 makes the remaining frontier explicit: Stage 5 has the highest answerable F_1 in that comparison, but Stage 4 has the strongest development-time over-

all F_1 and the lowest unsupported-answer rate among those later controller and learner checkpoints, and none of those checkpoints reaches the nominal 20% unsupported-answer budget on development. These findings support the claim that explicit abstention supervision, support verification, post-hoc calibration, and direct constrained learning are all informative, but they also show that verifier repair alone has limited headroom, that the support signal remains the weaker part of the pipeline even after calibration, that the first jointly trained learner is still too permissive, that the first adaptive revision behaves mostly like a slightly more conservative version of the same trade-off rather than a breakthrough, and that even the first explicit risk-budgeted action formulation does not yet generalize its validation-side safety gains to the development split. Taken together, the current stages are best read as evidence that neither post-hoc modular tuning, a first-pass support-constrained objective, a first adaptive revision, nor the first risk-budgeted action learner yet solves the end-to-end answer-support-abstain trade-off cleanly. The executed Stage 4 baseline makes that picture sharper rather than murkier: calibration improves the control signals, but the first calibrated controller still does not beat the earlier modular gate on development, and the later direct learners still fail to produce a better groundedness frontier.

1 Introduction

Large language models can produce answers that are fluent and plausible while still being unsupported by the available evidence. For this project, the main concern is not hallucination as a vague catch-all label, but unsupported answering delivered as if it were reliable. That framing matters because improving average accuracy does not automatically reduce overconfident unsupported outputs.

This paper takes a deliberately narrow starting point. Instead of attempting to solve hallucination in general, we study a controlled evidence-grounded question answering task in which a system must either answer or abstain. The current implementation uses SQuAD 2.0 [9] with one question and one evidence passage as input. Stage 1 compares two training regimes:

1. a baseline trained only on answerable examples, and
2. an abstain-aware variant trained to treat unsupported cases as a first-class target behavior.

Stage 2 then adds a verifier over question, passage, and predicted answer in order to estimate whether an answer is actually supported before it is shown to the user. After the original verifier proves too permissive, Stage 2.5 retrains that same verifier with Stage 1 QA predictions as hard negatives in order to narrow the train-versus-deployment mismatch. Stage 3 keeps the Stage 1 and Stage 2 models fixed and calibrates their score outputs so that later control stages can act on better-behaved confidence signals rather than raw overconfident values. Stage 4 has now also been run once as a fixed controller over those calibrated signals, and Stages 5, 6, and 7 have been run once each as successive direct learning alternatives. Taken together, these stages are no longer just incremental modules; they are also diagnostic experiments. Stage 1 shows that abstention supervision can change behavior but does not fully preserve answer quality, Stage 2 shows that support is learnable but only weakly changes end-to-end behavior when used as a permissive post-hoc gate, Stage 2.5 shows that hard-negative verifier repair reduces permissiveness only slightly, Stage 3 shows that both score streams are overconfident before calibration, Stage 4 shows that a validation-feasible fixed controller over the calibrated signals is still almost indistinguishable from the earlier Stage 2 gate on development, the first Stage 5 run shows that joint training can recover answerable-question quality while still failing to control unsupported answers well enough, the first Stage 6 run shows that a simple adaptive revision can make the learner only slightly more conservative without actually improving the final trade-off, and the first Stage 7 run shows that an explicit

risk-budgeted action formulation can satisfy its validation budgets yet still fail to carry that safety gain to the development split. The final comparison notebook over Stage 1 and Stages 4–7 sharpens this one step further: the direct learners do not merely "fail to win" one by one, but instead cluster in a higher-answerable yet worse-grounded region of the development trade-off surface. This sequence suggests that the remaining bottleneck is not only model quality, but also objective mismatch: the system is still trained and selected mostly around answer correctness, then patched afterward with separate support and abstention controls, and even the current direct learners still do not learn a risk signal that generalizes tightly enough beyond validation-time selection.

The central hypothesis is simple: explicit abstention supervision should reduce unsupported answers without causing an unacceptable collapse in answer quality on answerable questions. That hypothesis is narrow enough to evaluate cleanly, but still meaningful as a first step toward a broader program on grounded answering, support verification, and calibrated uncertainty. The next immediate question is whether a learned support signal can recover some of the remaining quality-versus-grounding gap without undoing the abstention gain from Stage 1. The current paper now motivates a second hypothesis for the next phase: post-hoc modular tuning has limited headroom in this setting, and neither the first support-constrained learner, its first adaptive revision, nor the first risk-budgeted action learner solves the problem yet. A better formulation should still treat evidence-grounded QA as a support-constrained learning problem rather than as answer prediction followed by separate filters, but it also has to make unsupported-answer costs bind more strongly than they do in the current Stage 5–7 runs. Concretely, the current comparison now keeps the same backbone and supervision source while pitting an executed calibrated modular controller against the already executed Stage 5, Stage 6, and Stage 7 direct-learning variants, so the paper can test whether the gain comes from changing the learning target rather than from adding yet another module.

The intended application setting is also narrow. KeelNet is most naturally suited to bounded, evidence-grounded QA scenarios in which answers should be tied to supplied documents and abstention is preferable to unsupported guessing, such as internal knowledge assistants, support systems over approved documentation, and policy or compliance QA. It is not intended as a general solution for open-domain truthfulness, balanced opinion generation, or fairness beyond the scope of the provided evidence.

This framing is also consistent with recent work arguing that hallucination persists partly because modern evaluation pipelines reward guessing more than appropriate uncertainty [5]. Our contribution is narrower and more operational: rather than explaining hallucination in general, Stage 1 tests whether an evidence-grounded QA system can be trained and evaluated so that abstention is treated as a useful target behavior instead of a failure by default, and Stage 2 tests whether a learned support verifier can improve downstream answer-versus-abstain decisions on top of that base system. Stage 2.5 then tests whether the original Stage 2 bottleneck was mainly a data problem by injecting hard QA-prediction negatives into verifier training. Stage 3 then asks whether those QA and support scores can be calibrated enough to serve as credible control variables in the next stage. If those stages succeed only partially, that is still informative: it supports a reformulation in which the current modular pipeline is compared against a direct support-constrained multi-head learner under matched data, backbone, and evaluation conditions, while also making clear that neither the first direct learner, nor its first adaptive follow-up, nor its first risk-budgeted action reformulation is automatically a win.

This LaTeX scaffold is intentionally written as a living project paper. Some sections already reflect stable design decisions, while others are structured to absorb final metrics, figures, and ablations once the experimental results are locked in.

2 Problem Setup

The current problem setting is evidence-grounded extractive question answering with a constrained binary action:

$$f(q, e) \rightarrow \{\text{answer span}, \textsc{abstain}\}, \tag{1}$$

where q is a question and e is a single evidence passage.

The Stage 1 scope is intentionally strict:

- Dataset: SQuAD 2.0 only.
- Input: one question and one evidence passage.
- Output: extractive answer span or ABSTAIN.
- Current implementation goal: reduce unsupported answers without sharply degrading performance on answerable questions.

This scope excludes retrieval, multi-hop reasoning, custom architectures, long-form generation, and broader claim-level verification. Those extensions are part of later roadmap stages, but they are out of scope for the current paper draft's core experimental claim.

We evaluate the task with metrics that separate utility from caution. For answerable examples, we track exact match and token-level F1. For unanswerable examples, we track unsupported answer rate and abstention quality. This makes it possible to study the trade-off between being useful and being appropriately cautious instead of collapsing everything into a single accuracy number.

3 Method

Stage 1 uses a standard extractive question answering architecture with a binary external decision: the system must either return an answer span from the passage or output ABSTAIN. The mechanism is intentionally simple. Rather than introducing a separate verifier or controller at this stage, the project asks whether abstention can already emerge from standard extractive QA supervision when unanswerable cases are represented correctly during training.

3.1 Encoder and QA heads

Let the input be a question q and a passage p. After tokenization, the model receives a single packed sequence of the form

$$x = [\text{CLS}], q, [\text{SEP}], p, [\text{SEP}]. \tag{2}$$

The encoder produces contextual token representations

$$h_1, h_2, \ldots, h_T = \text{Enc}_\theta(x), \tag{3}$$

where T is the tokenized sequence length. In the default setup, the encoder is initialized from the DistilBERT checkpoint `distilbert-base-uncased`. The implementation goes through the Hugging Face `AutoModelForQuestionAnswering` interface so the backbone can be swapped later.

The question answering head is the standard two-logit extractive head. For each token representation h_t, the model predicts:

$$s_t = w_s^\top h_t + b_s, \tag{4}$$

$$e_t = w_e^\top h_t + b_e, \tag{5}$$

where s_t is the start logit and e_t is the end logit for token t.

For a candidate span (i, j) with $i \leq j$, the raw span score is

$$S(i, j) = s_i + e_j. \tag{6}$$

This is the core deep learning mechanism of Stage 1: the encoder learns a contextual representation of the question-passage pair, and the two linear heads learn where the answer should begin and end.

3.2 Training targets

The project compares two training modes under the same architecture, preprocessing, and optimization settings.

Baseline mode. The baseline is trained only on answerable examples. This makes it a standard extractive QA model: every training case has a supervised span target inside the passage.

Abstain-aware mode. The abstain-aware model is trained on both answerable and unanswerable examples. For answerable examples, the targets are the gold start and end token positions, just as in the baseline. For unanswerable examples, both the start and end targets are mapped to the CLS position.

This matters because it turns abstention into a first-class supervised outcome without changing the external architecture. The model is not given a separate abstention head. Instead, it learns to use the score of the CLS token as a null or no-answer signal.

The optimization objective is the standard averaged cross-entropy loss over start and end targets:

$$\mathcal{L} = \frac{1}{2}\Big(\mathrm{CE}(s, y^{\mathrm{start}}) + \mathrm{CE}(e, y^{\mathrm{end}})\Big), \quad (7)$$

where y^{start} and y^{end} are the gold token indices. For unanswerable examples in the abstain-aware setting,

$$y^{\mathrm{start}} = y^{\mathrm{end}} = c, \quad (8)$$

with c denoting the CLS index.

3.3 Long-context handling

Because passages can exceed the maximum encoder length, Stage 1 uses the standard sliding-window extractive QA preprocessing strategy. Each example may be expanded into multiple overlapping features with a fixed maximum sequence length and document stride. During training, each feature receives span supervision relative to its own window. During evaluation, the pipeline keeps the metadata needed to map feature-level predictions back to the original example.

This implementation detail is important in deep learning terms because the model is trained over token windows rather than over raw passages directly. The final example-level prediction is therefore an aggregation over feature-level start and end logits.

3.4 Inference mechanism

At inference time, the model produces start and end logits for every feature of an example. Stage 1 then computes two competing quantities:

1. the best valid answer span score across all candidate spans and all windows, and
2. the best null score, defined as the sum of the start and end logits at the CLS position.

If the best span is $(i^\star, j^\star)$, then the best answer score is

$$S^\star_{\mathrm{span}} = \max_{i,j \in \mathcal{V}} S(i,j), \quad (9)$$

where $\mathcal{V}$ contains only valid spans that satisfy the ordering and maximum-answer-length constraints.

The null score is

$$S_\varnothing = s_c + e_c, \quad (10)$$

again using the CLS position c.

The baseline model always outputs the highest-scoring span. The abstain-aware model compares the null score against the best span through a margin:

$$\Delta = S_\varnothing - S^\star_{\mathrm{span}}. \quad (11)$$

The final decision rule is:

$$\hat{y} = \begin{cases} \textsc{abstain}, & \text{if } \Delta > \tau, \\ \text{best span}, & \text{otherwise}, \end{cases} \quad (12)$$

where τ is a validation-selected threshold.

Conceptually, this means Stage 1 does not learn a symbolic rule for when to refuse. It learns a representation in which the no-answer state competes directly against answer spans in the same scoring space.

3.5 Threshold selection

The abstention threshold is not fixed by hand in the default evaluation path. Instead, the code performs a simple search over a bounded range on the validation split. For each candidate threshold, the system post-processes predictions and measures Stage 1 metrics. The current implementation selects the threshold by prioritizing abstain F_1, with answerable F_1 used as a secondary tie-breaker.

This choice reflects the Stage 1 research question. The project is not only asking whether the model can refuse more often; it is asking whether it can reduce unsupported answers while preserving usefulness on answerable examples.

3.6 What Stage 1 proves

From a deep learning perspective, Stage 1 is a representation and decision control test. It asks whether a standard extractive QA network, with no additional verifier and no retrieval pipeline, can be pushed toward better answer-versus-abstain behavior purely through:

- different supervision over answerable versus unanswerable cases,
- a shared encoder and span head,
- the CLS-based null mechanism, and
- thresholded post-processing over the learned logit space.

If this works, then later stages can build on a meaningful base mechanism instead of introducing extra components before the basic abstention behavior is understood.

3.7 Stage 2 verifier and support gate

Stage 2 keeps the Stage 1 abstain-aware QA model fixed and adds a second model that judges whether a proposed answer is actually supported by the passage. This makes the overall system modular rather than jointly end-to-end: Stage 1 proposes an answer-or-ABSTAIN decision, and Stage 2 decides whether a predicted answer should be trusted.

Verification examples. The verifier is trained as a binary classification problem built from the same underlying SQuAD 2.0 splits. For each answerable training example, the pipeline creates one positive verification record using a gold answer and one or more negative records using sampled spans from the same passage that do not overlap strongly with the gold answer. For each unanswerable example, the pipeline creates only negative records by sampling unsupported spans from the passage. The result is a support-label dataset over tuples (q, p, a), where q is the question, p is the passage, and a is a candidate answer.

Stage 2.5 hard-negative repair. The original Stage 2 verifier is trained mostly on gold answers and sampled unsupported spans. That is useful for learning a first support signal, but it leaves a mismatch between training time and deployment time, because the verifier is evaluated on actual Stage 1 QA predictions rather than on random span negatives. Stage 2.5 narrows that mismatch by adding verifier examples built from the Stage 1 abstain-aware model's answered predictions. For each answered prediction $\hat{a}$, the pipeline adds a verifier record $(q, p, \hat{a})$ and labels it as supported only if the source example is answerable and $\hat{a}$ reaches the same answer-overlap support criterion used downstream. This makes the verifier see harder positives and harder negatives that are much closer to the answers it must judge at inference time.

Verifier input and model. The verifier formats the first sequence as a short hypothesis-like query,

$$x^{(1)} = \texttt{Question:}\quad q\ \texttt{\textbackslash n Answer:}\quad a, \tag{13}$$

and uses the passage as the second sequence,

$$x^{(2)} = p. \tag{14}$$

These paired sequences are fed into a standard transformer sequence classifier initialized from the same `distilbert-base-uncased` checkpoint family as Stage 1, but with a two-class support head. If h_{CLS} is the final pooled representation, the verifier computes support logits

$$z = Wh_{\text{CLS}} + b, \tag{15}$$

followed by a softmax that estimates

$$P_\phi(\text{supported} \mid q, p, a). \tag{16}$$

Training uses standard cross-entropy over the binary support labels.

Inference and gating. At evaluation time, only examples that Stage 1 already labeled as `answer` are sent to the verifier. If the Stage 1 system abstains, the final decision remains ABSTAIN. If Stage 1 answers, Stage 2 attaches a support score $s_{\text{sup}} \in [0, 1]$ and applies a second threshold:

$$\hat{y}_{\text{final}} = \begin{cases} \text{answer}, & \text{if } \hat{y}_{\text{qa}} = \text{answer and } s_{\text{sup}} \geq \eta, \\ \text{ABSTAIN}, & \text{otherwise.} \end{cases} \tag{17}$$

Here η is a validation-selected support threshold. This means the Stage 2 system is a cascaded decision process: the QA model first decides whether answering is plausible at all, and the verifier then decides whether that proposed answer appears passage-supported.

Threshold search and Stage 2 objective. The support threshold is selected on the validation split by prioritizing support F_1, then gated overall F_1, and then lower unsupported-answer rate. This reflects the Stage 2 question directly: not only whether support is learnable, but whether a learned support signal can improve the final answer-versus-ABSTAIN decision. Stage 2.5 keeps the same downstream selection rule so that any gain can be attributed to verifier supervision rather than to a different controller.

3.8 Revised problem formulation: support-constrained selective prediction

The early stages motivate a reformulation of the project objective. Up to Stage 3, the system is intentionally modular: the QA model is trained first, the verifier is trained second, and calibration is fit after both models are frozen. That modular structure is useful for diagnosis, but it also leaves the final decision objective implicit. The system is still learned mainly for answer prediction and then adjusted afterward with verifier thresholds and calibrated score transformations.

The empirical pattern from Stages 1, 2, 2.5, and 3 suggests that this post-hoc modular path has limited headroom in the current setting. Stage 1 improves abstention behavior but sacrifices answerable-question quality, Stage 2 learns support but changes end-to-end QA only marginally, and Stage 3 improves score reliability without changing the underlying decision objective. This motivates a cleaner formulation in which evidence-grounded QA is treated as a constrained selective prediction problem rather than as answer extraction followed by separate filters.

Let the final system choose either an answer a or abstention $\varnothing$ for each question-passage pair (q, p). Define a task utility $U(\hat{y}, y, p)$ that rewards a correct supported answer, penalizes an unsupported answer more strongly, and penalizes unnecessary abstention more mildly. The next-stage learning problem can then be written abstractly as

$$\max_{\theta} \quad \mathbb{E}\left[U(\hat{y}_\theta, y, p)\right] \tag{18}$$

$$\text{s.t.} \quad R_{\text{unsup}}(\theta) \leq \tau_{\text{u}}, \tag{19}$$

$$R_{\text{overabs}}(\theta) \leq \tau_{\text{a}}, \tag{20}$$

where R_{unsup} is unsupported-answer risk and R_{overabs} is unnecessary-abstention risk. In words: the system should maximize answer utility while staying within an explicit groundedness budget.

A practical surrogate for this idea is a Lagrangian objective

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{ans}} + \lambda_{\text{u}} \mathcal{L}_{\text{unsup}} + \lambda_{\text{a}} \mathcal{L}_{\text{overabs}}, \tag{21}$$

where $\mathcal{L}_{\text{ans}}$ is the standard answer loss, $\mathcal{L}_{\text{unsup}}$ penalizes unsupported answering, and $\mathcal{L}_{\text{overabs}}$ penalizes abstaining when a supported correct answer was available. This does not require a claim about a new universal learning law. Its contribution is narrower and more defensible: it makes the project's true target explicit instead of treating support and abstention only as post-hoc corrections to answer accuracy.

A feasible deep-learning instantiation for Stage 5. The Stage 5 comparison does not require a new backbone or a new dataset. It can be realized with the same encoder family already used in earlier stages, but with a different training objective and a tighter coupling between an-

swer extraction, support, and abstention. Concretely, let $x = [\text{CLS}], q, [\text{SEP}], p, [\text{SEP}]$ and let a shared encoder produce contextual states $h_1, \ldots, h_T = \text{Enc}_\theta(x)$. Stage 5 then augments the standard extractive QA heads with two additional binary decision heads so that answer extraction, support, and abstention are learned jointly from the same representation. The model exposes:

- span logits $z_t^{\text{s}} = w^{\text{s}\top} h_t + b^{\text{s}}$ and $z_t^{\text{e}} = w^{\text{e}\top} h_t + b^{\text{e}}$ for answer start and end positions,
- an abstention logit $z^{\varnothing} = w^{\varnothing\top} h_{\text{CLS}} + b^{\varnothing}$, and
- a support logit $z^{\text{sup}} = w^{\text{sup}\top} h_{\text{CLS}} + b^{\text{sup}}$.

Define $p_{\text{abs}} = \sigma(z^{\varnothing})$ and $p_{\text{sup}} = \sigma(z^{\text{sup}})$. Let $y_{\text{sup}} \in \{0, 1\}$ denote whether the gold answer is supported by the provided passage, and let $y_{\text{keep}} \in \{0, 1\}$ denote whether the final system should emit an answer rather than ABSTAIN. In the current single-passage setting, the two coincide: $y_{\text{keep}} = y_{\text{sup}} = 1$ for answerable supported examples and $y_{\text{keep}} = y_{\text{sup}} = 0$ for unanswerable or unsupported cases. The model then defines a differentiable keep probability

$$p_{\text{keep}} = (1 - p_{\text{abs}})\, p_{\text{sup}}, \tag{22}$$

which operationalizes the idea that an answer should survive only when the model is simultaneously unwilling to abstain and confident that the answer is supported.

A practical Stage 5 surrogate objective is then

$$\mathcal{L}_{\text{stage5}} = \mathcal{L}_{\text{span}} + \lambda_{\text{k}} \mathcal{L}_{\text{keep}} + \lambda_{\text{sup}} \mathcal{L}_{\text{sup}}, \tag{23}$$

$$\mathcal{L}_{\text{span}} = y_{\text{keep}} \left[\text{CE}(z^{\text{s}}, i^*) + \text{CE}(z^{\text{e}}, j^*) \right], \tag{24}$$

$$\mathcal{L}_{\text{keep}} = -\alpha y_{\text{keep}} \log p_{\text{keep}} - \beta \left(1 - y_{\text{keep}}\right) \log(1 - p_{\text{keep}}), \tag{25}$$

$$\mathcal{L}_{\text{sup}} = \text{BCE}(p_{\text{sup}}, y_{\text{sup}}), \tag{26}$$

with $\beta > \alpha$ so that unsupported answering is penalized more heavily than unnecessary abstention. This is the deep-learning form of the constrained objective above: $\mathcal{L}_{\text{span}}$ preserves standard extractive QA supervision, $\mathcal{L}_{\text{keep}}$ learns the final answer-versus-ABSTAIN decision under asymmetric cost, and $\mathcal{L}_{\text{sup}}$ keeps the support head explicitly grounded. The important difference from the modular pipeline is that these terms are optimized jointly through the shared encoder instead of being trained in separate stages and combined only at inference time.

This formulation is feasible in the current project for two reasons. First, every term is differentiable with respect to the model outputs, so the whole objective can be optimized end to end with standard gradient-based training. Second, the required supervision is already available or can be derived from the current task setup: gold spans provide (i^*, j^*), unanswerable examples supervise abstention, and the support target can be constructed from the same answerable-versus-unsupported labeling logic already used in Stages 2 and 3. So Stage 5 is not a speculative architectural leap. It is a controlled change in the learning objective that can be tested under matched backbone, data, and evaluation conditions.

3.9 Stage 6 as Stage 5 v2: answer-conditioned adaptive constraint balancing

The current Stage 5 learner establishes that direct support-constrained learning can move the answer-versus-grounding trade-off, but the first run also shows a specific weakness: answerable-question quality improves faster than unsupported-answer control. That suggests that the next learning revision should not be treated as an entirely new stage family. It should be treated as a second-generation version of Stage 5 itself. Stage 6 is therefore best defined as *Stage 5 v2*: the same backbone and dataset regime, but with better-aligned support supervision and adaptive constraint handling.

The key architectural change is that support should become *answer-conditioned*. Instead of predicting support only from the question-passage representation, let the Stage 6 learner score a small set of candidate spans $a_1, \ldots, a_K$ proposed by the span head. For each candidate, construct $x_k = [\text{CLS}], q, [\text{SEP}], a_k, [\text{SEP}], p, [\text{SEP}]$ and compute a candidate-conditioned representation

$g_k = \mathrm{Enc}_\theta(x_k)$. Stage 6 then predicts:

- a support logit z_k^{sup} for whether candidate a_k is actually supported by the passage,
- a keep logit z_k^{keep} for whether the final system should emit a_k rather than ABSTAIN, and
- the original span logits from the shared extractive QA head.

This follows the selective prediction literature's core lesson that the selection mechanism should be learned end to end and tied to the actual prediction target, not only to a generic confidence proxy [3, 7, 11, 12].

The supervision should also become harder and more realistic than in the current Stage 5 setup. Positive candidates are gold supported spans. Negative candidates should include model-predicted unsupported spans, lexical confounders from the same passage, and answerable-but-incorrect spans that are plausible enough to survive the base span head. This turns support learning into a candidate-level discrimination problem rather than a passage-level answerability signal. It also aligns Stage 6 more closely with the calibration literature in QA, which shows that confidence becomes useful only when it separates correct from incorrect answers at the answer level [4, 10, 13].

Under this design, a practical Stage 6 objective is

$$\mathcal{L}_{\mathrm{stage6}} = \mathcal{L}_{\mathrm{span}} + \lambda_{\mathrm{sup}}\mathcal{L}_{\mathrm{sup}}^{\mathrm{cand}} + \lambda_{\mathrm{keep}}\mathcal{L}_{\mathrm{keep}}^{\mathrm{cand}} + \lambda_{\mathrm{rank}}\mathcal{L}_{\mathrm{rank}} \tag{27}$$

$$+ \mu_{\mathrm{u}}\left[\hat{R}_{\mathrm{unsup}} - \tau_{\mathrm{u}}\right]_+ + \mu_{\mathrm{a}}\left[\hat{R}_{\mathrm{overabs}} - \tau_{\mathrm{a}}\right]_+ \tag{28}$$

where $\mathcal{L}_{\mathrm{sup}}^{\mathrm{cand}}$ is candidate-level support supervision, $\mathcal{L}_{\mathrm{keep}}^{\mathrm{cand}}$ learns answer versus ABSTAIN over candidates, $\mathcal{L}_{\mathrm{rank}}$ encourages supported gold spans to outrank hard unsupported candidates, and the final two terms penalize explicit budget violations for unsupported answering and over-abstention. The quantities $\hat{R}_{\mathrm{unsup}}$ and $\hat{R}_{\mathrm{overabs}}$ are batch-level surrogates for the two risks. This follows the constrained-optimization view that real operating budgets should be represented directly in the learning problem rather than only enforced after training by threshold search [2].

The balancing coefficients should also become adaptive instead of fixed. One option is to update the dual variables online,

$$\mu_{\mathrm{u}} \leftarrow \left[\mu_{\mathrm{u}} + \eta\left(\hat{R}_{\mathrm{unsup}} - \tau_{\mathrm{u}}\right)\right]_+, \tag{29}$$

$$\mu_{\mathrm{a}} \leftarrow \left[\mu_{\mathrm{a}} + \eta\left(\hat{R}_{\mathrm{overabs}} - \tau_{\mathrm{a}}\right)\right]_+, \tag{30}$$

so that unsupported-answer pressure automatically increases when the learner violates the groundedness budget. Another option is to adapt the task-loss weights themselves by uncertainty-based weighting or gradient balancing [1, 6]. In either case, the main point is the same: Stage 6 should not merely reuse the Stage 5 loss with different constants. It should let the optimization react when the learner becomes too permissive or too refusal-heavy.

So Stage 6 is no longer just a vague "adaptive balancing" idea. It is a concrete, paper-facing Stage 5 revision: candidate-conditioned support scoring, model-mined hard negatives, and adaptive constraint multipliers under matched backbone and data conditions. If it works, the gain is attributable to better-aligned supervision and constraint handling rather than to extra model scale or a looser evaluation setup.

3.10 Stage 7: risk-budgeted action learning over candidates

The current Stage 6 design still leaves one structural weakness in place: it scores candidate answers independently and then applies a threshold to the best one. That means the final system is still not learning the full decision problem in one step. It learns candidate scores first and only afterward decides whether any answer should survive. Because Stage 6 remained too weak in the first run, the next step is not another small loss-weight adjustment. It changes the decision formulation itself.

Stage 7 is best defined as a *risk-budgeted action learner*. Given a question-passage pair (q, p), let $\mathcal{A}(q, p) = \{a_1, \ldots, a_K, \varnothing\}$ be the action set consisting of the top K candidate spans

plus an explicit ABSTAIN action $\varnothing$. For each answer candidate a_k, Stage 7 computes an answer-conditioned representation g_k as in Stage 6. It then predicts two separate quantities:

$$u_k = f_{\text{util}}(g_k), \tag{31}$$

$$c_k = f_{\text{risk}}(g_k), \tag{32}$$

where u_k is a utility score for answering with a_k and c_k is a risk score for how likely that answer is to be unsupported. The abstain action receives its own learned utility score $u_\varnothing$, while its unsupported-answer cost is fixed to zero by construction.

The final decision is made over actions, not over candidates plus a later gate:

$$\hat{a} = \arg\max_{a \in \mathcal{A}(q,p)} \left[u(a) - \lambda_{\text{u}} c(a) \right], \tag{33}$$

with $c(\varnothing) = 0$. To prevent obviously unsafe answers from slipping through, Stage 7 also keeps a hard deployment shield:

$$\hat{y} = \begin{cases} \text{ABSTAIN}, & \text{if } c(\hat{a}) > \rho, \\ \hat{a}, & \text{otherwise}, \end{cases} \tag{34}$$

where ρ is a validation-selected maximum tolerated answer risk. This makes abstention an explicit competitor in the action space rather than a thresholded afterthought.

The training signal should likewise separate utility from risk. For answerable examples, the correct action is the supported gold answer span. For unanswerable examples, the correct action is $\varnothing$. Let y^{act} denote the gold action and let $\hat{R}_{\text{unsup}}$ and $\hat{R}_{\text{overabs}}$ be batch-level surrogates for unsupported answering and unnecessary abstention. A practical Stage 7 objective is then

$$\mathcal{L}_{\text{stage7}} = \mathcal{L}_{\text{act}} + \lambda_{\text{sup}} \mathcal{L}_{\text{sup}}^{\text{cand}} + \mu_{\text{u}} \left[\hat{R}_{\text{unsup}} - \tau_{\text{u}} \right]_+ \tag{35}$$

$$+ \mu_{\text{a}} \left[\hat{R}_{\text{overabs}} - \tau_{\text{a}} \right]_+ + \lambda_{\text{tail}} \mathcal{L}_{\text{tail}}, \tag{36}$$

where $\mathcal{L}_{\text{act}}$ is cross-entropy over the $K+1$-way action space and $\mathcal{L}_{\text{tail}}$ upweights the highest-risk answered negatives so the learner focuses on the confident unsupported answers that matter most operationally. The key shift is that the model no longer asks only "which span looks best?" It asks "which action maximizes utility while staying within the risk budget?"

The dual variables are also updated explicitly rather than only through a single negative-class weight. In the spirit of constrained optimization and safe RL, Stage 7 increases μ_{u} when unsupported-answer cost exceeds budget and increases μ_{a} when the system becomes too refusal-heavy [2, 8]. This does not turn the project into reinforcement learning: the setting remains supervised and one-step. But it does import the most useful safe-RL lesson into the current QA regime, namely that reward-like utility and safety-like cost should be optimized together under an explicit budget rather than only reconciled by threshold search after training.

Under this formulation, Stage 7 is not "more Stage 6." It is the first stage that treats grounded QA as a genuine utility-versus-risk action problem. If it succeeds, the gain is attributable to a better decision formulation: explicit abstain action, explicit risk head, explicit budgets, and explicit dual updates. The first Stage 7 run now makes the diagnostic value concrete: the selected risk threshold satisfies both validation budgets, but the same operating point still yields 34.62% unsupported-answer rate on the held-out development split. So the remaining bottleneck is no longer just the lack of an explicit action space or explicit dual updates. It is that the learned risk signal still does not generalize tightly enough beyond validation-time selection.

3.11 Stage 8 Hybrid and Stage 8.2: final learned architecture

Stage 8 asks a narrower question than Stage 7: if the Stage 5 learner is already good at proposing plausible spans, and Stage 4 is already good at producing calibrated groundedness signals, should the final system learn only the last decision layer instead of retraining the entire QA stack? That leads to two closely related variants. Stage 8 Hybrid restores calibrated control on

top of the frozen Stage 5 answer engine. Stage 8.2 keeps the same frozen answer engine, but replaces the fixed final controller with a small risk-budgeted action network that also consumes the calibrated Stage 4 support signal. Among the learned systems, Stage 8.2 is the final chosen architecture because it is the cleanest executed comparison of learned action selection plus calibrated evidence support under a clean final split.

Frozen candidate generator. Stage 8.2 does not retrain the Stage 5 DistilBERT QA encoder. Instead, the Stage 5 checkpoint is used as a frozen candidate generator. For each example, it produces a bounded candidate set $\mathcal{A}(q,p) = \{a_1, \ldots, a_K, \varnothing\}$, where the K answer actions are the top-ranked Stage 5 spans and $\varnothing$ is the explicit abstain action. This is important architecturally: the heavy transformer encoder is reused as a proposal network, while the new learning problem is pushed into a much smaller action-selection model.

Candidate features. Each candidate a_k is converted into a compact feature vector $x_k \in \mathbb{R}^{12}$. The current implementation uses Stage 5 span score, gap to the best span, margin to the next span, keep probability, calibrated Stage 4 support probability, Stage 5 abstain probability, optional Stage 6 controller probability, answer length, normalized rank, question-answer lexical overlap, keep-versus-support disagreement, and a keep-uncertainty term. In other words, Stage 8.2 does not ask the final controller to rediscover span semantics from scratch. It receives an already-structured candidate description containing both answer quality and groundedness cues.

Two-head candidate encoder plus abstain head. The final controller is a shallow MLP rather than another large transformer. For each candidate feature vector x_k, the model computes

$$h_k = \mathrm{MLP}(x_k), \tag{37}$$

$$u_k = w_u^\top h_k + b_u, \tag{38}$$

$$r_k = \sigma(w_r^\top h_k + b_r), \tag{39}$$

where u_k is a utility logit for answering with candidate a_k and r_k is a risk probability for that answer being unsupported. The candidate encoder is a two-layer GELU MLP with dropout. To model abstention as a first-class action, the network also pools the candidate hidden states with mean and max pooling and sends the concatenated summary through a separate abstain head:

$$u_\varnothing = f_{\mathrm{abs}}([\mathrm{mean}(h_1, \ldots, h_K); \max(h_1, \ldots, h_K)])\,. \tag{40}$$

So the final decision layer is explicitly group-aware: abstention depends on the whole candidate set, not just on the top span.

Risk-adjusted action selection. Stage 8.2 keeps the Stage 7 decision logic but improves the support features feeding it. Each candidate action is scored by subtracting a dual-weighted risk penalty from its utility logit:

$$s_k = u_k - \lambda_{\mathrm{u}} r_k. \tag{41}$$

The abstain action gets its own logit $s_\varnothing = u_\varnothing - \lambda_{\mathrm{a}}$, where λ_{a} is the over-abstention dual. The final action distribution is then

$$\pi(a \mid q,p) = \mathrm{softmax}([s_1, \ldots, s_K, s_\varnothing]). \tag{42}$$

At inference time the system first keeps only candidates whose predicted risk falls below the selected threshold ρ. If calibrated Stage 4 support is available, it also applies a hard support shield so that candidates below the calibrated support floor are removed before the final answer-or-ABSTAIN choice.

Why this is the final chosen learned model. Stage 8.2 is the paper's final learned architecture because it cleanly combines the three most useful ingredients discovered earlier:

1. Stage 5 provides a strong frozen answer proposal space,
2. Stage 4 provides calibrated support as a groundedness feature rather than only as a post-hoc gate, and
3. Stage 7 provides an explicit action-space learner with separate utility, risk, and abstain components.

This makes Stage 8.2 the most faithful end-to-end test of the paper's deep learning hypothesis: that grounded QA should be modeled as learned action selection over candidate answers under calibrated evidence support and explicit safety budgets. It is *not* claimed as the strongest overall operating point in the final results. That distinction remains important: Stage 4 is still the best executed overall trade-off, while Stage 8.2 is the cleanest final learned architecture. Figure 1 summarizes that flow in the same candidate-to-action order used by the implementation.

3.12 Experimental design for the remaining comparison

Under this revised formulation, the next comparison should be deliberately simple and controlled. The same backbone, dataset split, preprocessing, and core QA setup should be kept fixed, while only the decision formulation changes. The intended comparison is therefore:

1. the current modular pipeline (Stage 1 abstention, Stage 2 support verification, and Stage 3 calibration),
2. a fixed constrained controller over the calibrated signals, and
3. a direct support-constrained learner with the same encoder family and preprocessing pipeline, trained under matched conditions,
4. a Stage 6 answer-conditioned adaptive learner, and
5. a Stage 7 risk-budgeted action learner with explicit abstain choice and explicit unsupported-answer and over-abstention budgets,
6. a Stage 8 Hybrid controller that restores calibrated control on top of the Stage 5 answer engine, and
7. a Stage 8.2 action learner that injects calibrated Stage 4 support back into the Stage 7 action formulation under a clean final split.

This comparison design is important because it isolates the real question: whether the main remaining gain comes from further module tuning, or from changing the learning objective itself. The direct-learning side of this comparison is now already executed through Stage 7, and the modular Stage 4 controller baseline is also now executed under matched conditions. Its near-null development result makes the comparison more concrete rather than less: the paper no longer needs to speculate about whether calibrated fixed control would help in principle, because the first such controller is now available as an empirical reference point.

3.13 Stage 3 confidence calibration

Stage 3 does not retrain the Stage 1 QA model or the Stage 2 verifier. Instead, it treats both of them as fixed score generators and asks whether their confidence values can be made more trustworthy through lightweight post-hoc calibration. The goal is not yet to change the final controller. The goal is to produce calibrated decision signals that later stages can trust when deciding whether to answer, abstain, or suppress a weakly supported answer.

QA decision calibration. For the Stage 1 abstain-aware model, Stage 3 reuses the same validation-selected abstention threshold τ and constructs a scalar QA decision score from the margin between the null score and the best answer-span score. If the QA system predicts ABSTAIN, the score is defined as

$$z_{\mathrm{qa}} = \Delta - \tau, \tag{43}$$

and if it predicts an answer, the sign is flipped so that larger values always mean *more confidence that the final QA decision was correct*:

$$z_{\mathrm{qa}} = \tau - \Delta. \tag{44}$$

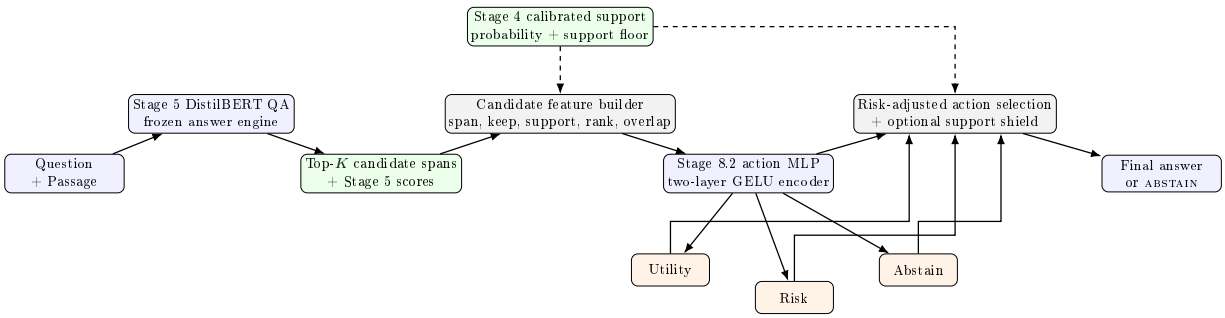

Figure 1: Compact left-to-right Mermaid-like LaTeX flowchart of the final chosen learned architecture, Stage 8.2. The Stage 5 DistilBERT checkpoint is frozen and used only to generate the top-K answer candidates. Stage 4 contributes calibrated support as both a candidate feature and an optional hard support shield. A shallow Stage 8.2 action network then predicts utility, risk, and abstain scores before making the final answer-or-ABSTAIN decision.

The binary QA calibration label is 1 if the final Stage 1 decision was correct under the decision-aware evaluation rule and 0 otherwise.

Support-score calibration. For Stage 2, only QA predictions that remain `answer` are passed to the support verifier, just as in the original Stage 2 pipeline. The verifier produces a support probability $p_{\text{sup}} = P_\phi(\text{supported} \mid q, p, a)$. Stage 3 first maps that probability back to a logit-like score through the inverse sigmoid:

$$z_{\text{sup}} = \log \frac{p_{\text{sup}}}{1 - p_{\text{sup}}}, \tag{45}$$

and then calibrates this score against a binary target indicating whether the predicted answer actually matches a gold answer strongly enough to count as supported.

Temperature scaling. Both score streams are calibrated separately with scalar temperature scaling. Given a raw score z and a learned temperature $T > 0$, the calibrated probability is

$$\tilde{p} = \sigma\left(\frac{z}{T}\right), \tag{46}$$

where $\sigma(\cdot)$ is the sigmoid function. The implementation searches over a bounded grid of temperatures on the validation split and picks the value that minimizes the Brier score, with expected calibration error (ECE) used as the tie-breaker. This is a deliberately simple post-hoc mechanism: the underlying QA and verifier parameters stay fixed, and only the sharpness of their confidence mapping changes.

Stage 3 metrics and artifacts. Stage 3 reports raw-versus-calibrated metrics for both QA and support scores on the held-out development split. The main reported quantities are ECE, adaptive ECE, maximum calibration error (MCE), Brier score, a threshold-gap measure, score-label correlation, and reliability diagrams. Because the same Stage 1 QA threshold and Stage 2 support threshold are kept fixed, Stage 3 should be read as a calibration study rather than as a new end-to-end controller. Its success criterion is narrower: reduce overconfidence and make the score distributions more faithful to empirical correctness so that later control stages have a better starting signal. In deep learning terms, Stage 2 tests whether a second learned representation space over (q, p, a) can act as a useful post-hoc controller on top of the Stage 1 answer space.

4 Experimental Protocol

The active experiments are run from the project notebooks and Python modules in the repository. The current implementation uses a compact extractive QA setup with a default DistilBERT backbone and SQuAD 2.0 preprocessing.

Dataset split. The training pipeline creates deterministic train, validation, and development splits from SQuAD 2.0. For the answer-only baseline, the training set is restricted to answerable examples. For the abstain-aware setting, both answerable and unanswerable cases are kept. For Stage 2, the verifier is trained on examples derived from the Stage 1 QA outputs so that each instance contains a question, a passage, a predicted answer, and a support label. For Stage 2.5, the verifier training set is augmented further with actual Stage 1 QA predictions as hard positives and hard negatives, labeled by the same answer-support criterion used at evaluation time. For Stage 3, the Stage 1 abstain-aware model and the original Stage 2 verifier are kept fixed, and calibration is fit post hoc on the validation split before raw and calibrated confidence behavior is reported on the development split. The current draft reports Stage 2.5 separately as a repair attempt because its end-to-end deltas are small enough that a second full calibration table would be mostly duplicative.

Evaluation. Model outputs are post-processed into final decisions and scored with:

- answerable exact match,
- answerable F1,
- decision-aware overall exact match,
- decision-aware overall F1,
- unsupported answer rate, and
- abstain precision, recall, and F1.

The decision-aware overall metrics give full credit to correct abstentions on unanswerable examples, so they do not penalize uncertainty by construction. For Stage 2, we additionally report support accuracy, precision, recall, and F1, together with supported-answer rate and contradiction rate for the verifier and its downstream gated QA behavior. The same verifier-side and gated QA metrics are reused for Stage 2.5 so that the hard-negative repair can be compared directly against the original Stage 2 pipeline. For Stage 3, we additionally report expected calibration error (ECE), adaptive ECE, maximum calibration error (MCE), Brier score, threshold-gap statistics, score-label correlation, and reliability diagrams for both the QA decision signal and the support signal. For the next comparison stage, the same core metrics should be kept while adding a controller-facing view: supported-answer rate among answered cases, answer rate, and the selected unsupported-answer budget.

Execution workflow. The current team workflow runs experiments from browser Google Colab while editing source code locally in VS Code. The notebook supports both hosted Colab and Colab connected to a local runtime; the Stage 1, Stage 2, Stage 2.5, and Stage 3 runs reported here used a local runtime backed by the project's WSL environment and GPU, while the executed Stage 4, Stage 6, and Stage 7 runs reported here used hosted Colab on an A100 GPU because later controller search, candidate generation, replay through the Stage 5 learner, and the action-learning pipeline were more memory intensive. Artifacts are stored under the project runtime directory, and code changes are synchronized through Git branches.

Current comparison design. The current experimental comparison holds the backbone, split, tokenizer, and evaluation protocol fixed while varying only the decision formulation. With the current Stage 4, Stage 5, Stage 6, and Stage 7 results in hand, the most informative full comparison is now:

1. the current modular pipeline (Stage 1 abstention, Stage 2 verifier, Stage 3 calibration),
2. a fixed constrained controller over calibrated QA and support signals, and
3. a jointly trained support-constrained multi-head learner that keeps the same encoder, tokenizer, split, and core QA supervision but

replaces post-hoc control with an answer-support-abstain objective,

4. a Stage 6 revision of that learner that makes support answer-conditioned, introduces hard candidate negatives, and adapts its unsupported-answer and over-abstention penalties during training,

5. a Stage 7 risk-budgeted action learner that turns candidate answers and abstention into one action space and optimizes utility under explicit unsupported-answer and over-abstention budgets.

That comparison is now instantiated on both sides: the modular path is executed through Stage 4, and the direct-learning path is executed through Stage 7. The Stage 4 controller is the intended calibrated-controller reference, even though its first run turns out to be almost identical to the original Stage 2 gate on development and therefore does not become the strongest modular checkpoint by a meaningful margin. Under this design, the key question is not whether one more module can be bolted on, but whether a direct constrained objective improves the final answer-support-abstain trade-off beyond what post-hoc modular tuning can achieve, and whether later learner revisions improve because of better aligned supervision rather than simple retuning. To make that comparison concrete at the end of the pipeline, we also aggregate the strongest completed checkpoints into one final-comparison notebook that produces a single table and trade-off plots over Stage 1 baseline and abstain plus Stages 4–7.

What to lock before the final paper. Before turning this draft into a submission-ready paper, we should freeze:

- the final model checkpoint choice,
- the exact QA, verifier, calibration, and controller-selection policies,
- the final metric tables and reliability figure,
- at least one qualitative error analysis slice, and

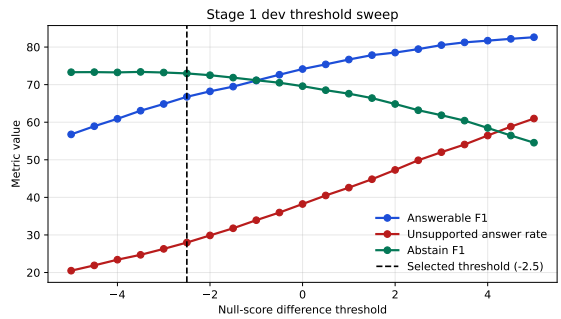

Figure 2: Threshold sweep for the abstain-aware model on the development split. The vertical line marks the validation-selected threshold of -2.5.

- the figure or table that best shows the utility-versus-groundedness trade-off for modular versus constrained-learning approaches.

5 Results

Table 1 is the core Stage 1 comparison. It reports one stable baseline run and one stable abstain-aware run under matched training settings.

The baseline is trained only on answerable examples, while the abstain-aware variant includes explicit supervision for unanswerable cases. For Stage 1, a win is defined conservatively: unsupported-answer rate should decrease, answerable performance should remain usable, and abstain quality should improve without the model collapsing into trivial refusal behavior. Under that conservative criterion, the current Stage 1 result is a qualified success. The abstain-aware model cuts unsupported-answer rate by 72.03 points, from 100.00% to 27.97%, and raises decision-aware overall F_1 by 27.05 points, from 42.35 to 69.40. At the same time, answerable F_1 drops by 18.07 points, from 84.82 to 66.76, so the improvement comes with a real answer-quality cost rather than a free gain.

Because the abstain-aware system uses a validation-selected threshold, the main table should be read together with a threshold sweep rather than as a single isolated operating point.

Table 1: Main Stage 1 comparison between the answer-only baseline and the abstain-aware model on the held-out development split.

Model	Answerable F1	Decision-aware Overall F1	Unsupported Rate	Abstain F1
Baseline	84.82	42.35	100.00	–
Abstain-aware	66.76	69.40	27.97	72.98

Threshold interpretation. Figure 2 shows the expected trade-off as the threshold moves toward less negative values. Across the dev sweep from -3.5 to -1.5, answerable F_1 rises from 63.06 to 69.47, but unsupported-answer rate also rises from 24.71% to 31.77%. Abstain F_1 changes more gently, moving from 73.39 to 71.86 across the same range. The selected threshold of -2.5 sits in the middle of this trade-off region: neighboring points at -3.0 and -2.0 have similar dev overall F_1 (69.29 and 69.17 versus 69.40 at -2.5), which suggests that the gain is not coming from a single brittle threshold setting.

Stage 1 takeaway. The main question for Stage 1 was whether explicit abstention supervision could correct the baseline's pathological always-answer behavior. On that question, the answer is yes. The baseline answers every unsupported example, while the abstain-aware model learns a meaningful abstain policy and achieves 72.98 abstain F_1 on the dev split. The cost is that answer extraction quality remains substantially worse than the baseline on answerable questions, which gives a clear target for Stage 2 and later stages: preserve the abstention gain while recovering answer quality.

Stage 2 support-verification result. Stage 2 keeps the Stage 1 abstain-aware QA model fixed and adds a verifier over question, passage, and predicted answer. The selected support threshold is 0.25. On the validation split, the verifier reaches 87.71 support F_1. On the held-out development split, it reaches 79.79 support F_1, with a 66.26% supported-answer rate and a 33.00% contradiction rate. When that support signal is used to gate the Stage 1 abstain-aware system, the end-to-end QA metrics move only slightly: decision-aware overall F_1 rises from 69.40 to 69.42 and unsupported-answer rate falls from 27.97% to 27.57%, while answerable F_1 slips from 66.76 to 66.41.

Stage 2.5 hard-negative repair. Stage 2.5 keeps the same Stage 1 abstain-aware base model and the same gated evaluation rule, but retrains the verifier after augmenting its training data with answered Stage 1 QA predictions as hard positives and hard negatives. In the current run, this adds 72,184 QA-derived verifier examples to the train split, 7,606 to validation, and 6,084 to the held-out dev-side verification set. The repair does make the verifier slightly less permissive: dev support precision rises from 66.62% to 67.05%, predicted-supported rate falls from 98.88% to 97.07%, and contradiction rate falls from 33.00% to 31.99%. The downstream gated unsupported-answer rate also drops from 27.57% to 26.95%. However, support F_1 barely changes (79.79 to 79.70), answerable F_1 falls from 66.41 to 65.62, and gated overall F_1 slips from 69.42 to 69.34. The selected support threshold also moves sharply upward, from 0.25 to 0.95, which indicates that the repaired verifier still needs an aggressive cutoff to avoid over-trusting answered predictions.

How to read the Stage 2 and Stage 2.5 outcome. The positive result is that the verifier does learn a meaningful support signal instead of collapsing completely. The limiting result is that neither the original verifier nor the hard-negative repair translates into a large downstream improvement. Stage 2 clearly overconfirms answers, and Stage 2.5 softens that tendency only slightly. The repaired verifier is directionally better on precision and contradiction rate, but the gains are too small to recover answerable quality or to produce a convincing end-

Table 2: Stage 2 and Stage 2.5 support verification on top of the Stage 1 abstain-aware QA system, evaluated on the held-out development split. Stage 2.5 retrains the verifier with Stage 1 QA predictions as hard negatives. All values are percentages except the selected support threshold.

	Verifier				End-to-end QA		
System	Thr.	Prec.	Rec.	F1	Ans. F1	Overall F1	Unsup. Rate
Stage 1 abstain QA	–	–	–	–	66.76	69.40	27.97
Stage 2 gated QA	0.25	66.62	99.43	79.79	66.41	69.42	27.57
Stage 2.5 hard-neg. gated QA	0.95	67.05	98.24	79.70	65.62	69.34	26.95

to-end jump. So Stage 2.5 should be read as a useful repair attempt rather than as a new stage that overturns the earlier diagnosis. It confirms that easy-negative training data was part of the problem, but not the whole problem. The larger bottleneck remains the same: later stages need stronger control over how support scores are used, or a different learning objective altogether.

Representative failure cases. The saved dev predictions make the current bottleneck concrete. On the unanswerable question *"What is France a region of?"*, the Stage 1 baseline answers *"Normandy"*, the Stage 1 abstain-aware model still answers *"Normandy"*, and Stage 2 assigns that answer a support score of 0.9998 and keeps the final decision as `answer`. This is a clean false-positive support error: the passage contains both *France* and *Normandy*, but the relation in the question is reversed relative to the evidence. By contrast, on the answerable question *"If input size is equal to n, what can respectively be assumed is the function of n?"*, the Stage 1 abstain-aware model correctly predicts *"the time taken"*, but Stage 2 assigns that answer a support score of only 0.0052 and gates it to ABSTAIN. This is the complementary false-negative support error: the verifier can over-penalize short abstract answers even when the QA model is already correct. Stage 2.5 reduces the aggregate permissiveness of the verifier slightly, but it does not remove this qualitative pattern. Taken together, these examples match the aggregate metrics: the current verifier is strong enough to learn support as a signal, but not yet selective enough to improve the final decision reliably.

Stage 3 calibration result. Stage 3 applies scalar temperature scaling to two score streams without retraining the underlying models: the Stage 1 answer-versus-ABSTAIN decision score and the Stage 2 support score. Both selected temperatures hit the top of the searched range, $T_{\text{qa}} = 5.0$ and $T_{\text{sup}} = 5.0$, which is itself a useful finding. It indicates that both score distributions were substantially too sharp before calibration, especially the verifier. On the dev split, the raw QA signal has mean confidence 0.9458 against actual decision accuracy 0.7002, while the raw support signal has mean confidence 0.9873 against actual support accuracy 0.6665. So before calibration, both modules are strongly overconfident rather than only slightly misaligned.

What improved. The QA signal responds well to this simple calibration step. Dev QA ECE falls from 0.2456 to 0.0753, MCE falls from 0.3289 to 0.1019, Brier score improves from 0.2631 to 0.2009, and the threshold-gap measure falls from 0.1122 to 0.0890. The support signal also improves materially: ECE drops from 0.3326 to 0.1708, adaptive ECE from 0.3247 to 0.1686, MCE from 0.8838 to 0.2883, Brier score from 0.3324 to 0.2483, and threshold gap from 0.1317 to 0.0894. Figure 3 shows the same pattern qualitatively. Both calibrated curves move closer to the diagonal, and the extreme near-certainty behavior of the raw support signal is softened substantially.

Table 3: Stage 3 calibration results on the held-out development split. The QA signal is the Stage 1 answer-versus-ABSTAIN decision score, and the support signal is the Stage 2 verifier score over answered cases only. Lower is better for ECE, adaptive ECE, MCE, Brier, and threshold gap; higher is better for correlation.

Signal	Setting	ECE	Adapt. ECE	MCE	Brier	Threshold Gap	Corr.
QA decision	Raw	0.2456	0.2456	0.3289	0.2631	0.1122	0.1909
QA decision	Calibrated	0.0753	0.0753	0.1019	0.2009	0.0890	0.2691
Support verifier	Raw	0.3326	0.3247	0.8838	0.3324	0.1317	0.0778
Support verifier	Calibrated	0.1708	0.1686	0.2883	0.2483	0.0894	0.1296

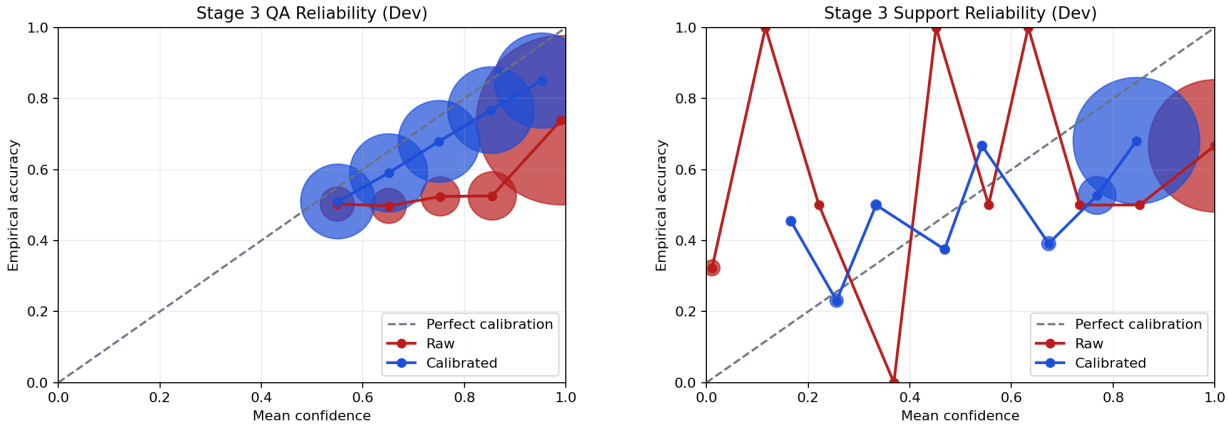

Figure 3: Stage 3 reliability diagrams on the development split. Left: the Stage 1 abstain-aware QA decision score. Right: the Stage 2 support-verifier score. In both cases, scalar temperature scaling moves the calibrated curve toward the diagonal, but the support signal remains visibly less reliable than the QA decision signal.

What did not improve enough. Stage 3 is not yet a new downstream controller, so the end-to-end answer and abstain metrics remain the Stage 2 ones at this stage. The key question is therefore not whether overall QA F_1 changed already, but whether the confidence signals became good enough to drive a stronger fixed policy in Stage 4. On that question, the answer is mixed. The calibrated QA signal now looks reasonably usable. The calibrated support signal is better than before, but it is still weaker than QA on every reported calibration metric and its correlation with the true support label remains low at 0.1296. So Stage 3 succeeds as a post-hoc calibration study, but it also confirms that support confidence remains the fragile part of the pipeline.

Stage 3 takeaway. The main finding from Stage 3 is not that temperature scaling solves grounded QA by itself. The main finding is that the earlier Stage 2 bottleneck was real: the verifier's scores were badly overconfident and needed flattening before they could be trusted by later stages. Simple scalar calibration already fixes much of that overconfidence, but the support signal still lags the QA decision signal after calibration. This sharpens the next-stage objective considerably: Stage 4 should not merely add another heuristic filter, but should combine the now-better-calibrated QA and support signals into a tighter unsupported-answer controller and test whether that finally produces a meaningful end-to-end gain. The later Stage 2.5 hard-negative repair slightly reduces verifier permissiveness, but it does not materially change this interpretation. Read together, Stages 1, 2,

2.5, and 3 therefore support a broader methodological point: modular post-hoc tuning can improve grounded QA behavior, but in this setup it does not yet close the objective gap between answer correctness, evidence support, and abstention quality. That is the empirical reason to compare future modular controllers against a direct support-constrained learning formulation rather than continuing to add modules without changing the learning target.

Stage 4 fixed-controller result. Stage 4 uses the calibrated QA and support signals from Stage 3 and searches a fixed controller under a nominal 20% unsupported-answer budget on validation. The selected validation configuration uses support threshold 0.40, QA threshold 0.40, joint threshold 0.45, and $\alpha = 0.90$. On the validation split, this configuration satisfies the budget with 14.30% unsupported-answer rate and reaches 72.43 decision-aware overall F_1. On the held-out development split, however, the controller reaches 66.41 answerable F_1, 69.41 decision-aware overall F_1, 27.59% unsupported-answer rate, 72.96 abstain F_1, 66.61% supported-answer rate among answered cases, and a 50.68% answer rate.

How to read the Stage 4 result. The key finding is that calibration clearly improves the quality of the confidence signals, but the first fixed controller does not turn those better signals into a better development-time decision rule. Relative to the original Stage 2 gated system, answerable F_1 is unchanged at 66.41, overall F_1 slips by 0.01 points from 69.42 to 69.41, unsupported-answer rate worsens by 0.02 points from 27.57% to 27.59%, and abstain F_1 also slips by 0.01 points. Supported-answer rate among answered cases and answer rate are likewise almost identical. So Stage 4 should be read as a useful null result: the calibrated controller is validation-feasible, but on development it is effectively indistinguishable from the simpler Stage 2 gate. That makes the modular story sharper, not weaker: calibration matters for score quality, but the first fixed controller still fails to leverage that better score behavior into a meaningful end-to-end gain.

Cross-stage final comparison. The final-comparison notebook aggregates the strongest completed Stage 1 and Stage 4–8 Hybrid artifacts into common trade-off plots and one normalized summary table. On the answerable-F_1-versus-unsupported frontier, those checkpoints separate into two clear regions. Stage 1 abstain QA and Stage 4 fixed control both sit near 66.4–66.8 answerable F_1 with roughly 27.6–28.0% unsupported-answer rate. By contrast, Stages 5–8 Hybrid sit higher, around 68.4–71.4 answerable F_1, but only at roughly 31.9–34.8% unsupported-answer rate. So the direct-learning family has not yet moved the frontier outward on the held-out development split; it has moved the system into a different operating regime. Within that final-comparison slice, Stage 4 is the strongest development-time overall trade-off, Stage 5 is the answerable-quality leader, and Stage 8 Hybrid is the strongest groundedness recovery among the later learned-controller variants without reclaiming the modular frontier. Across all executed checkpoints in Table 4, including Stage 2 and Stage 2.5, no method reaches the nominal 20% unsupported-answer budget on the development split.

Stage 5 standalone result. Stage 5 is intended to compare a jointly trained support-constrained learner against the executed modular controller family. That comparison is now available directly: Stage 4 provides the intended calibrated-controller baseline, and the original Stage 2 gate remains the slightly stronger modular checkpoint on development by 0.01 overall F_1. In the current run, the selected keep threshold is 0.65. On the held-out development split, the learner reaches 71.40 answerable F_1, 68.28 decision-aware overall F_1, 34.84% unsupported-answer rate, 71.17 abstain F_1, 62.40% supported-answer rate among answered cases, and a 58.38% answer rate. The run is also evaluated under a nominal 20% unsupported-answer budget, which it misses by

Table 4: Executed modular and direct-learning checkpoints in the final comparison. Most rows report held-out development results. Stage 3 is omitted because it recalibrates the QA and support scores but does not define a separate end-to-end decision system. Stage 4 is the calibrated fixed-controller baseline, but the strongest executed modular checkpoint remains the original Stage 2 gate by a negligible margin. Stage 6 is evaluated against the Stage 5 learner from which it derives candidate features, Stage 7 is evaluated against the same Stage 5 learner plus the Stage 6 controller features, Stage 8 Hybrid restores calibrated control on top of the Stage 5 answer engine, and Stage 8.2 reports its clean final result on the untouched test split.

System	Split	Answerable F1	Decision-aware Overall F1	Unsupported Rate
Stage 1 abstain QA	dev	66.76	69.40	27.97
Stage 2 gated QA	dev	66.41	69.42	27.57
Stage 2.5 hard-neg. gated QA	dev	65.62	69.34	26.95
Stage 4 fixed controller	dev	66.41	69.41	27.59
Stage 5 support-constrained learner	dev	71.40	68.28	34.84
Stage 6 adaptive learner	dev	69.70	67.62	34.45
Stage 7 risk-budgeted action learner	dev	69.95	67.66	34.62
Stage 8 hybrid controller	dev	68.39	68.27	31.86
Stage 8.2 action + calibrated support	test	70.06	68.16	33.73

14.84 points.

What Stage 5 improved and what it worsened. The positive result is clear: the support-constrained learner substantially recovers answerable-question quality. Relative to the Stage 1 abstain-aware model, answerable F_1 rises by 4.64 points, from 66.76 to 71.40. Relative to the Stage 2.5 hard-negative gated system, it rises by 5.78 points, from 65.62 to 71.40. However, the groundedness side moves in the wrong direction. Unsupported-answer rate rises from 27.97% in Stage 1 to 34.84% in Stage 5, and from 26.95% in Stage 2.5 to 34.84%. That increase is large enough that overall F_1 falls below every previously executed modular checkpoint, including the 69.41 reached by the calibrated Stage 4 controller and the slightly higher 69.42 reached by the original Stage 2 gated system.

How to read the current Stage 5 finding. This first Stage 5 run is therefore a useful negative result rather than a successful replacement for the modular pipeline. Changing the learning target does matter: the learner moves the trade-off strongly and recovers answerable extraction quality that the abstain-aware and verifier-gated systems had given up. But the current objective and thresholding setup do not yet bind unsupported answering tightly enough. So the direct learner is not automatically safer just because support and abstention are trained jointly. What it can claim already is sharper: the first direct support-constrained objective exposes a different failure mode than the modular pipeline, namely that answer quality can be recovered faster than unsupported-answer control.

Stage 6 adaptive revision result. Stage 6 is designed as a Stage 5 revision rather than as a new standalone pipeline: it reuses the trained Stage 5 learner to generate candidate features, then learns an adaptive balancer over those candidate-level signals. The fairest immediate comparison is still Stage 6 against Stage 5, but the run can now also be read against the executed modular baselines, including Stage 4. In the current run, the selected candidate threshold is 0.50. On the held-out development split, Stage 6 reaches 69.70 answerable F_1, 67.62 decision-aware overall F_1, 34.45% unsupported-answer rate, 71.12 abstain F_1, 62.41% supported-answer rate among answered cases, and a 57.77% answer rate. Under the same nominal 20% unsupported-answer budget used for Stage 5, it still misses the target by

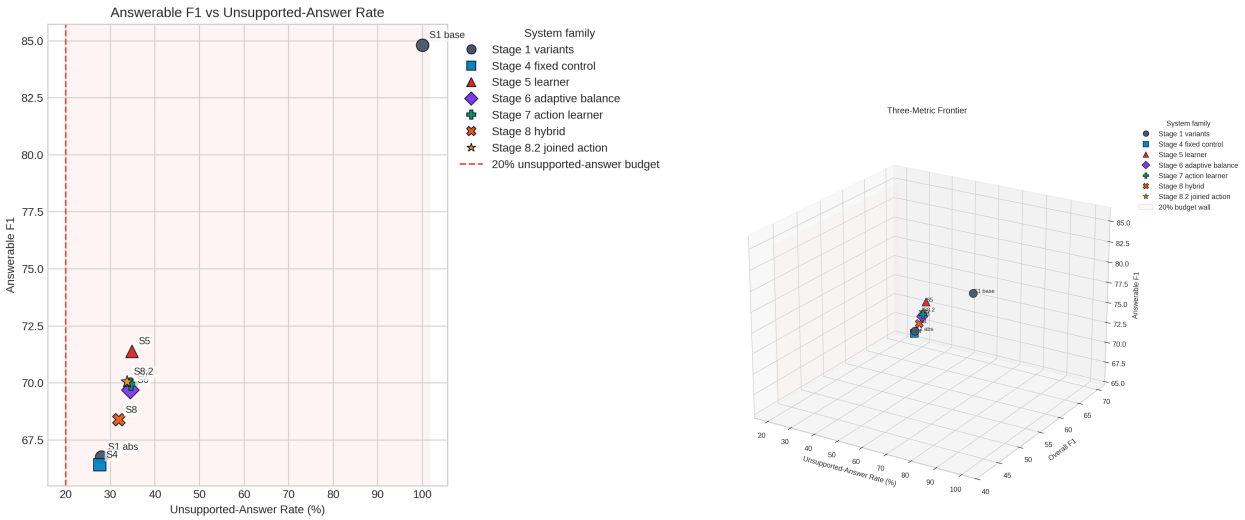

Figure 4: Notebook-generated final comparison views built from the completed artifacts. Left: the main answerable-F_1-versus-unsupported-answer frontier, where the Stage 8.2 marker is kept as a clean-split test reference and the learned family rises on answerable quality only by moving into a riskier unsupported-answer regime. Right: a three-metric view over unsupported-answer rate, decision-aware overall F_1, and answerable F_1. Read together, the plots show the same two-regime pattern: the modular side remains safer, while the later learned variants recover more answerable-question quality without yet reaching a better groundedness frontier.

14.45 points.

What Stage 6 changed and what it did not. Relative to Stage 5, the adaptive revision makes the system only slightly more conservative. Answer rate falls from 58.38% to 57.77%, and unsupported-answer rate falls by 0.39 points, from 34.84% to 34.45%. But the gain is too small to matter at the system level: supported-answer rate among kept answers is effectively unchanged, moving only from 62.40% to 62.41%, while answerable F_1 falls by 1.70 points and overall F_1 falls by 0.66 points. So the first adaptive learner trims answering slightly without materially improving groundedness.

How to read the current Stage 6 finding. The first Stage 6 run is therefore a second informative negative result rather than a breakthrough. It shows that adding candidate-level adaptive balancing on top of the current Stage 5 pipeline is not sufficient by itself. The gap is no longer only that the first direct learner is too permissive; it is also that the first adaptive correction changes behavior too weakly to recover the unsupported-answer budget or beat the simpler Stage 5 learner. What it can claim already is narrower and still useful: the current Stage 6 mechanism behaves mostly like a slightly more cautious Stage 5, not yet like a new frontier point on the utility-versus-groundedness tradeoff.

Stage 7 risk-budgeted action result. Stage 7 changes the formulation more aggressively than Stage 6. Instead of thresholding independently scored candidates, it treats candidate answers and ABSTAIN as one action space and learns separate utility and risk signals under explicit unsupported-answer and over-abstention budgets. In the current run, the selected risk threshold is 0.85. On the validation split, that operating point satisfies both target budgets: unsupported-answer rate is 19.22% against a 20% maximum, over-abstention rate is 12.92% against a 20% maximum, overall F_1 is 73.06, and answerable F_1 is 69.19. On the held-out development split, however, the same operating point reaches 69.95 answerable F_1, 67.66 decision-aware overall F_1, 34.62% unsupported-answer

rate, 71.11 abstain F_1, 62.28% supported-answer rate among answered cases, 57.99% answer rate, and 18.57% over-abstention rate.

What Stage 7 changed and what it did not. Relative to Stage 6, Stage 7 improves answerable F_1 only slightly, from 69.70 to 69.95, and overall F_1 only marginally, from 67.62 to 67.66. At the same time, unsupported-answer rate worsens slightly, from 34.45% to 34.62%, supported-answer rate among answered cases slips from 62.41% to 62.28%, and answer rate rises slightly from 57.77% to 57.99%. So the action learner does not win simply by refusing more often. Its over-abstention rate stays within the nominal 20% budget, but the unsupported-answer budget still fails on development by 14.62 points. Relative to Stage 5, Stage 7 is slightly less permissive (34.62% versus 34.84% unsupported-answer rate), but it still gives back 1.45 points of answerable F_1 and 0.62 points of overall F_1.

How to read the current Stage 7 finding. The first Stage 7 run is therefore a third informative negative result rather than the long-sought clean direct-learning win. The positive result is that the stronger formulation really does solve the *validation* problem it was designed for: explicit ABSTAIN, explicit risk, and explicit budgets are enough to select an operating point that satisfies both validation constraints. The limiting result is that this gain does not transfer to the held-out development split, where unsupported-answer rate remains essentially unchanged from the Stage 5–6 regime. So the current bottleneck is no longer just the lack of an explicit action space. It is the lack of a risk signal whose supported-versus-unsupported separation generalizes beyond validation-time threshold selection.

Stage 8 Hybrid result. Stage 8 returns to a modular combination: it keeps Stage 5 as the answer engine and adds calibrated Stage 4 control signals back on top. In the current run, the selected candidate threshold is 0.20. On the held-out development split, Stage 8 reaches 68.39 answerable F_1, 68.27 decision-aware overall F_1, 31.86% unsupported-answer rate, 71.71 abstain F_1, 64.27% supported-answer rate among answered cases, and a 54.91% answer rate.

What Stage 8 Hybrid changed and what it did not. Relative to the Stage 5–7 family, the hybrid is meaningfully more conservative. Unsupported-answer rate falls by 2.98 points relative to Stage 5, by 2.59 points relative to Stage 6, and by 2.76 points relative to Stage 7. That groundedness gain also lifts overall F_1 above the Stage 5–7 regime, with a 0.61-point improvement over Stage 7. But the recovery is still incomplete: answerable F_1 remains 3.01 points below Stage 5, and the hybrid still trails the calibrated Stage 4 controller by 1.14 overall F_1 and 4.27 points of unsupported-answer rate. So restoring calibrated control helps, but it does not yet reclaim the modular frontier.

Stage 8.2 clean-split action-learner result. Stage 8.2 keeps Stage 5 as the answer engine, injects calibrated Stage 4 support into the Stage 7 action learner, and reports its final checkpoint on a truly untouched test split. In the current run, the selected risk threshold is 0.70. On validation, that operating point satisfies the nominal unsupported-answer budget with 19.31% unsupported-answer rate and 72.98 overall F_1. On the final test split, however, it reaches 70.06 answerable F_1, 68.16 decision-aware overall F_1, 33.73% unsupported-answer rate, 71.09 abstain F_1, 63.28% supported-answer rate among answered cases, and a 56.72% answer rate. So the cleaned formulation reproduces the same central pattern under stricter evaluation hygiene: the controller can satisfy the budget on validation, but the supported-versus-unsupported separation still does not transfer tightly enough to the final held-out split.

Final chosen learned architecture. The paper's final chosen learned architecture is therefore Stage 8.2, not because it is the strongest final operating point, but because it is the cleanest executed deep-learning formulation of the

full hypothesis. It freezes the Stage 5 DistilBERT answer engine as a proposal network, rescoring the top candidate spans with a shallow action model that receives both answer-quality features and calibrated Stage 4 support features. That action model learns separate utility and risk heads for each candidate, plus a pooled abstain head over the whole candidate set. This architecture is the closest executed system to a true support-aware selective predictor: the final answer-or-ABSTAIN choice is learned directly over a compact action space, under explicit unsupported-answer budgets, while still preserving a modular calibrated support signal. The empirical result is still sobering. Figure 4 shows that the architecture moves the learned family toward better answerable F_1, but not yet to a safer overall frontier than Stage 4.

6 Discussion and Limitations

The current project deliberately favors scope control over breadth. That is a strength for the first paper draft, but it also creates limitations that should be stated clearly.

Current limitations.

- The active evidence now covers Stage 1, Stage 2, the Stage 2.5 hard-negative verifier repair, Stage 3, one executed Stage 4 fixed-controller baseline, one standalone Stage 5 run, one Stage 6 adaptive follow-up, and one Stage 7 risk-budgeted action run.
- The task uses a single provided evidence passage rather than retrieved evidence.
- The action space is intentionally narrow and does not yet cover richer support labels or multi-hop settings.
- The current results will reflect one controlled dataset setup rather than a broad claim about hallucination in general.

Intended deployment scope. The current approach is best interpreted as a method for high-trust, document-bounded QA, not as a general-purpose opinion or advice generator. Its most plausible deployment settings are ones in which the evidence source is explicit, the answer should remain short and verifiable, and abstaining is preferable to guessing. If the provided evidence is incomplete, outdated, or systematically biased, the method may still produce a well-grounded but incomplete answer, so groundedness should not be confused with neutrality or full real-world balance.

How to interpret the current result. The current evidence supports two related but different claims. First, the Stage 1 run shows that explicit abstention supervision and abstention-aware evaluation can strongly reduce unsupported answers in a simple extractive QA system. Second, the Stage 2 run shows that support verification is learnable on top of that abstain-aware base. Third, the Stage 2.5 repair shows that making verifier supervision harder does reduce permissiveness slightly, but only modestly. Fourth, the Stage 3 run shows that both the QA and support scores were indeed overconfident and that simple post-hoc temperature scaling can reduce that miscalibration substantially. However, the calibrated support signal remains noticeably weaker than the QA signal. Fifth, the Stage 4 run shows that a fixed controller over those calibrated signals is validation-feasible but still almost identical to the original Stage 2 gate on development. Sixth, the first standalone Stage 5 run shows that a jointly trained answer-support-abstain learner can recover answerable-question quality, but in its current form it also raises unsupported-answer rate enough to lower overall F_1 and miss its own unsupported-answer budget. Seventh, the first Stage 6 run shows that an adaptive revision of that learner reduces unsupported-answer rate only slightly and still misses the same budget, while giving back both answerable F_1 and overall F_1 relative to Stage 5. Eighth, the first Stage 7 run shows that an explicit utility-versus-risk action formulation can satisfy both unsupported-answer and over-abstention budgets on validation, yet

still miss the unsupported-answer budget badly on the held-out development split. The result should therefore be read as a trade-off improvement plus three useful intermediate controls plus one failed-to-break-through repair attempt plus three informative direct-learning failure modes, not yet as a generally better grounded QA model. The strongest interpretation is narrower: Stages 1, 2, 2.5, 3, and 4 support the usefulness of modular post-hoc controls, even though the first calibrated controller does not beat the earlier gate, while the first Stage 5, Stage 6, and Stage 7 runs show that direct constrained learning has promise but is not yet superior in this setup.

Why this stage is still useful. Even under its narrow scope, the reported stages now test several defensible claims with clean supervision and interpretable failure modes. Stage 1 shows that abstention can be taught. Stage 2 shows that support can be predicted, but also reveals the remaining downstream bottleneck clearly. Stage 2.5 then shows that the bottleneck is not explained away by a simple hard-negative data repair. Stage 3 then confirms that this bottleneck was not only architectural but also statistical: the raw confidence scores were too sharp to trust directly. After calibration, the QA signal becomes reasonably well behaved, while support still remains the noisier control variable. Stage 4 then makes the next point concrete: the calibrated controller can satisfy the unsupported-answer budget on validation, but on development it ends up effectively matching the original Stage 2 gate rather than improving on it. Stage 5 then shows that a joint learner can move the answer-quality side of the frontier, but not yet the groundedness side. Stage 6 sharpens that diagnosis further: the adaptive revision does make the system slightly more conservative, but the change is too weak to improve supported-answer quality or overall downstream performance. Stage 7 sharpens it again in a different way: the stronger action-space formulation does solve the validation-time constraint problem, but the unsupported-answer reduction does not generalize to development.

What the final comparison makes visible. The final-comparison notebook turns that diagnosis into a concrete frontier shape rather than a sequence of isolated stage summaries. On the development split, the modular checkpoints occupy a relatively tight region around 69.3–69.4 overall F_1 and about 27% unsupported-answer rate, while the current Stage 5–7 direct learners occupy a different region with higher answerable F_1 but lower overall F_1 and roughly 34–35% unsupported-answer rate. So the present direct-learning family has not yet produced a better Pareto frontier. It has produced a different trade-off regime whose gains are real on answerable questions, but whose groundedness costs are still too large on held-out data.

Legacy plan and revision. The original roadmap after Stage 1 was to move directly from Stage 2 support verification into Stage 4 unsupported-confidence control, while treating Stage 3 calibration as supporting work and deferring retrieval to a later realism check. That legacy plan assumed that once support became learnable, the raw verifier scores would already be good enough to drive downstream control. Stage 2 changed that assumption. On the dev split, support recall reaches 99.43%, but support precision is only 66.62%, the verifier predicts support for 98.88% of answered cases, and the gated system improves overall F_1 by only 0.03 points. Stage 2.5 sharpens the diagnosis rather than reversing it. With hard QA-prediction negatives, support precision rises only to 67.05%, the predicted-supported rate falls only to 97.07%, contradiction rate falls from 33.00% to 31.99%, and gated unsupported-answer rate falls from 27.57% to 26.95%, while gated overall F_1 slips slightly from 69.42 to 69.34. Stage 3 then validates the revised plan directly. Both fitted temperatures saturate at 5.0, dev QA ECE drops from 0.2456 to 0.0753, and dev support ECE drops from 0.3326 to 0.1708, but support still remains less calibrated than QA after the fix. So the long-term goal stays the same,

and the immediate plan is now clearer than before: Stage 4 does operate on calibrated signals rather than raw ones, and its first run sharpens the diagnosis again: the selected controller satisfies the unsupported-answer budget on validation, but on development the result is almost unchanged relative to the original Stage 2 gate. So the remaining gap is no longer simply "add a controller over calibrated signals." It is either to design a stronger controller family or to make the learning objective itself carry more of the groundedness burden.

From modular tuning to constrained learning. The current results also motivate a deeper paper-level pivot. They do not show that modular design is misguided: each module adds useful information. What they do show is that separately trained and separately tuned modules do not automatically solve the end-to-end answer-support-abstain problem. The remaining gap is therefore best interpreted as an objective mismatch. The system still learns mainly for answer extraction and only later receives support and abstention corrections. That observation motivates a revised next-stage comparison in which the current modular pipeline is treated as the baseline and a support-constrained learning objective is treated as the new method. Under that framing, the earlier experiments become evidence for the limitation of post-hoc modular control in this setting rather than failed attempts that should be discarded. The key methodological point is that this Stage 5 learner does not need to be larger or more exotic than the modular baseline. It can use the same encoder family and the same data while changing only the training target: answer extraction, support, and abstention are optimized jointly by a shared-encoder multi-head model instead of in three separate stages. That makes the next comparison much cleaner. The first standalone Stage 5 run shows both why that comparison matters and why it is not automatically favorable to the new objective. Answerable F_1 rises to 71.40, but unsupported-answer rate rises to 34.84% and overall F_1 falls to 68.28. So the joint learner does change the frontier, but not yet in the desired direction for grounded answering. The current Stage 5 objective appears too permissive: it recovers answer extraction quality faster than it suppresses weakly supported answers. That is still useful evidence, because it narrows the next question. Stage 6 turns that question into an explicit test. In the current run, unsupported-answer rate falls only from 34.84% to 34.45%, supported-answer rate among answered cases stays flat at about 62.4%, answerable F_1 falls from 71.40 to 69.70, and overall F_1 falls from 68.28 to 67.62. So the first adaptive revision behaves mostly like a slight conservatizing wrapper around the same permissive answer frontier. The paper therefore no longer needs to ask whether direct constrained learning, first-pass adaptivity, or explicit action-space reformulation changes behavior at all; it now needs to ask why the current risk estimates still fail to carry their validation-side gains into the development split. That makes the next revision much more concrete than the original roadmap suggested. It should not be treated as another generic adaptive add-on. The first Stage 7 run already provides the relevant decision formulation: an explicit risk-budgeted action learner with separate utility and risk scores, explicit ABSTAIN as an action, harder candidate negatives, and budget-driven dual updates. The recent selective-prediction literature reinforces that move, because it argues that learned abstention becomes more reliable when the selection signal is tied directly to the actual prediction and coverage trade-off [3, 7, 11, 12], while the constrained-optimization and multitask-balancing literature gives a natural way to make unsupported-answer budgets bind during training rather than only after threshold search [1, 2, 6], and the safe-RL literature contributes the useful design lesson that utility and safety cost should be optimized jointly under explicit budgets even when the final implementation remains supervised rather than sequential [8]. The Stage 7 artifact makes one more diagnostic point concrete: the saved model ends with an unsupported-answer dual of 0.0 and an over-abstention dual of 3.26. That pattern is con-

sistent with a learner whose adaptive pressure concentrated far more on refusal control than on suppressing unsupported answers, which in turn helps explain why over-abstention stays within budget while unsupported-answer rate still generalizes poorly on development.

Next paper-facing extensions. With Stage 1, Stage 2, Stage 2.5, Stage 3, Stage 4, and first runs of Stages 5, 6, and 7 now in place, the most natural paper extensions are:

- strengthening the Stage 4 controller family beyond the current near-null result so that the calibrated modular baseline is not limited to a validation-feasible but development-flat operating point,
- rerunning Stage 5 against that modular baseline directly with a support-constrained learning objective trained under matched conditions,
- strengthening Stage 6 as a Stage 5 revision with sharper answer-conditioned support scoring, harder candidate negatives, and adaptive unsupported-answer balancing that moves more than the current 0.39-point unsupported-rate reduction,
- analyzing why Stage 7's validation-feasible risk control fails to transfer to development, especially through candidate-risk calibration, dual-variable dynamics, and support-generalization error,
- treating further verifier-side repair as supporting work rather than as the main paper contribution if the modular baseline is still too weak,
- testing retrieval noise, and
- comparing the Stage 6 adaptive learner and the Stage 7 risk-budgeted learner against both the simpler Stage 5 learner and the stronger fixed modular baseline.

7 Conclusion

This draft paper frames KeelNet as a staged research program for evidence-grounded question answering with controlled abstention. The current reported contribution now covers Stage 1, Stage 2, a Stage 2.5 hard-negative verifier repair, Stage 3, a first executed Stage 4 fixed controller, a first standalone Stage 5 run, a first Stage 6 adaptive follow-up, and a first Stage 7 risk-budgeted action run under a controlled SQuAD 2.0 setting: first an answer-only baseline versus an abstain-aware alternative, then a learned support verifier on top of the abstain-aware system, and finally a post-hoc calibration step over the QA and support scores, followed by an initial jointly trained support-constrained learner, its first adaptive revision, and its first action-space reformulation.

The current evidence is encouraging but mixed in the right way for a staged project. Stage 1 substantially improves unsupported-answer behavior and decision-aware overall performance, but it still gives up too much answerable-question quality. Stage 2 then shows that support verification is learnable, yet the present gating policy is too permissive to create a large end-to-end gain. Stage 2.5 confirms that this bottleneck is not solved by a simple verifier data repair: hard negatives make the verifier slightly less permissive, but they do not produce a decisive downstream improvement. Stage 3 confirms that this permissiveness was accompanied by serious overconfidence in both score streams and shows that simple scalar temperature scaling can reduce that miscalibration substantially, although support remains the weaker signal even after calibration. Stage 4 then adds a useful null result: the first calibrated fixed controller satisfies the unsupported-answer budget on validation, but on the development split it reaches 69.41 overall F_1 and 27.59% unsupported-answer rate, which is effectively indistinguishable from the original Stage 2 gate at 69.42 and 27.57%. Stage 5 then adds a useful negative result: the first support-constrained learner recovers answerable-question quality up to 71.40 answerable F_1, but unsupported-answer

rate rises to 34.84% and overall F_1 falls to 68.28, so the current direct objective is still too permissive. Stage 6 then adds a second useful negative result: the first adaptive revision lowers unsupported-answer rate only slightly, from 34.84% to 34.45%, but answerable F_1 falls to 69.70 and overall F_1 falls to 67.62, so the current adaptive controller does not yet beat the simpler Stage 5 learner. Stage 7 then adds a third useful negative result: the first explicit risk-budgeted action learner satisfies both validation budgets at its selected threshold, yet on the development split it still reaches 34.62% unsupported-answer rate, with 69.95 answerable F_1 and 67.66 overall F_1. That means the project has cleared four useful bars: abstention supervision works, support prediction is viable, post-hoc calibration improves the quality of the control signals, and a validation-feasible calibrated controller can be selected from those signals. It has also run four useful negative tests: making the verifier harder to train does help a little, but not enough to remove the modular bottleneck by itself, and making the learner joint does change the trade-off, but not yet in a way that improves groundedness, while making that learner adaptive only conservatizes it slightly without producing a better frontier point, and making the learner action-based and explicitly risk-budgeted still does not solve the generalization gap between validation-side safety control and development-side unsupported answering. At the same time, the results sharply motivate the next stages, especially tighter unsupported-answer control, because the remaining question is no longer whether confidence can be calibrated at all, but whether calibrated QA and support signals can finally be combined into a materially better downstream decision policy, and whether a better-constrained Stage 5–7 family can do the same end to end. The final cross-stage comparison now makes the current frontier especially clear: Stage 5–7 shift the system toward higher answerable-question quality, but on the development split they still occupy an inferior groundedness regime with lower overall F_1 and higher unsupported-answer rate than the stronger modular checkpoints. No executed checkpoint yet reaches the nominal 20% unsupported-answer budget on development. That motivates a more explicit next-step formulation for the paper: use the existing modular pipeline as a strong baseline, then compare it against a jointly trained support-constrained multi-head learner that optimizes answer extraction, support, and abstention under explicit unsupported-answer and over-abstention costs while keeping the same encoder family and core dataset setup. The natural follow-up after those first learners is now also clear. The current Stage 6 run should be treated as the first concrete second-generation version of Stage 5, not as an unrelated later branch, and the current Stage 7 run should be treated as the first explicit test of whether a risk-budgeted action space fixes the Stage 5–6 bottleneck. Their current role is diagnostic: Stage 6 shows that answer-conditioned balancing, hard candidate features, and adaptive penalties are directionally sensible but still too weak in the present formulation, and Stage 7 shows that explicit utility-risk action selection can make the budgets bind on validation without making them transfer on development. So the next contribution can no longer be framed as "add Stage 6" or "design Stage 7" in the abstract. It has to be framed as learning a risk signal that generalizes: tighten the unsupported-answer constraint enough that adaptive and action-based learning improve groundedness rather than merely reshuffling the same Stage 5–7 frontier.

References

[1] Zhao Chen, Vijay Badrinarayanan, Chen-Yu Lee, and Andrew Rabinovich. Gradnorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 794–803, 2018. URL https://proceedings.mlr.press/v80/chen18a.html.

[2] Andrew Cotter, Heinrich Jiang, Maya Gupta, Serena Wang, Taman Narayan,

Seungil You, and Karthik Sridharan. Optimization with non-differentiable constraints with applications to fairness, recall, churn, and other goals. *Journal of Machine Learning Research*, 20(172):1–59, 2019. URL https://jmlr.org/papers/v20/18-616.html.

[3] Yonatan Geifman and Ran El-Yaniv. Selectivenet: A deep neural network with an integrated reject option. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 2151–2159, 2019. URL https://proceedings.mlr.press/v97/geifman19a.html.

[4] Zhengbao Jiang, Jun Araki, Haibo Ding, and Graham Neubig. How can we know when language models know? on the calibration of language models for question answering. *Transactions of the Association for Computational Linguistics*, 9:962–977, 2021. doi: 10.1162/tacl_a_00407. URL https://aclanthology.org/2021.tacl-1.57/.

[5] Adam Tauman Kalai, Ofir Nachum, Santosh S. Vempala, and Edwin Zhang. Why language models hallucinate, 2025. URL https://arxiv.org/abs/2509.04664.

[6] Alex Kendall, Yarin Gal, and Roberto Cipolla. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 7482–7491, 2018. URL https://openaccess.thecvf.com/content_cvpr_2018/html/Kendall_Multi-Task_Learning_Using_CVPR_2018_paper.html.

[7] Ziyin Liu, Zhikang Wang, Paul Pu Liang, Ruslan Salakhutdinov, Louis-Philippe Morency, and Masahito Ueda. Deep gamblers: Learning to abstain with portfolio theory. In *Advances in Neural Information Processing Systems 32*, 2019. URL https://papers.nips.cc/paper_files/paper/2019/hash/0c4b1eeb45c90b52bfb9d07943d855ab-Abstract.html.

[8] Zuxin Liu, Zhepeng Cen, Vladislav Isenbaev, Wei Liu, Steven Wu, Bo Li, and Ding Zhao. Constrained variational policy optimization for safe reinforcement learning. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 13644–13668, 2022. URL https://proceedings.mlr.press/v162/liu22b.html.

[9] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don't know: Unanswerable questions for SQuAD. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 784–789, 2018. doi: 10.18653/v1/P18-2124. URL https://aclanthology.org/P18-2124/.

[10] Chenglei Si, Chen Zhao, Sewon Min, and Jordan Boyd-Graber. Re-examining calibration: The case of question answering. In *Findings of the Association for Computational Linguistics: EMNLP 2022*, pages 2814–2829, 2022. doi: 10.18653/v1/2022.findings-emnlp.204. URL https://aclanthology.org/2022.findings-emnlp.204/.

[11] Tejas Srinivasan, Jack Hessel, Tanmay Gupta, Bill Yuchen Lin, Yejin Choi, Jesse Thomason, and Khyathi Chandu. Selective "selective prediction": Reducing unnecessary abstention in vision-language reasoning. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 12935–12948, 2024. doi: 10.18653/v1/2024.findings-acl.767. URL https://aclanthology.org/2024.findings-acl.767/.

[12] Hiyori Yoshikawa and Naoaki Okazaki. Selective-LAMA: Selective prediction for confidence-aware evaluation of language models. In *Findings of the Association for Computational Linguistics: EACL 2023*, pages 2017–2028, 2023. doi: 10.18653/v

1/2023.findings-eacl.150. URL https://aclanthology.org/2023.findings-eacl.150/.

[13] Shujian Zhang, Chengyue Gong, and Eunsol Choi. Knowing more about questions can help: Improving calibration in question answering. In *Findings of the Association for Computational Linguistics: ACL-IJCNLP 2021*, pages 1958–1970, 2021. doi: 10.18653/v1/2021.findings-acl.172. URL https://aclanthology.org/2021.findings-acl.172/.